RNS Trust ®

RNS Institute of Technology

Autonomous Institute of Technology Affiliated to VTU
Accredited with NAAC A+ Grade

BCY685, Project Phase 1

REVIEW 1 on March 12th, 2026, Thursday

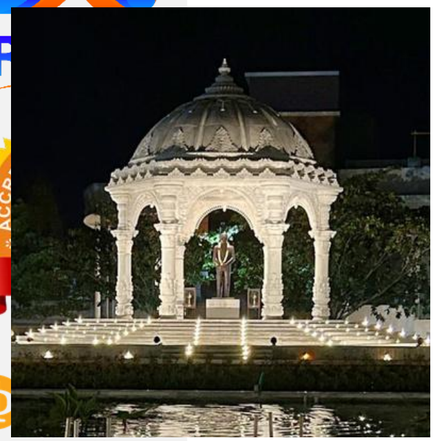
AI Powered Vulnerability Detection and Mitigation System

TEAM 3

Nithyashree R	1RN23CY035
Kavindra Nishod	1RN23CY023
Dev Kukreja	1RN23CY014

Guide

Dr. Manohar P
Dept of - CSE-CY



Bangalore Chapter

Tribute to Our Founder

Dr. R N Shetty
Founder
1928 - forever

RNSIT

RNS INSTITUTE OF TECHNOLOGY
An Autonomous Institute under VTU
Accredited with NAAC A+ Grade

Agenda

- Introduction
- Problem Statement and Objectives
- Literature Review
- Methodology
- Resources/Requirements
- Expected Outcomes
- Applications
- References

Introduction

Cybersecurity in Modern Software Ecosystems

- Modern applications rely heavily on open-source libraries and external packages
- Large software projects may include hundreds of dependencies
- These dependencies can introduce security vulnerabilities into the software supply chain
- Vulnerabilities are tracked globally as Common Vulnerabilities and Exposures (CVEs)

Challenge:

Managing vulnerabilities in complex software ecosystems has become a critical cybersecurity concern.

Introduction

Existing Vulnerability Detection Tools

Examples include:
- OWASP Dependency-Check
- Snyk
- GitHub Dependabot
- Container vulnerability scanners

Key Limitations

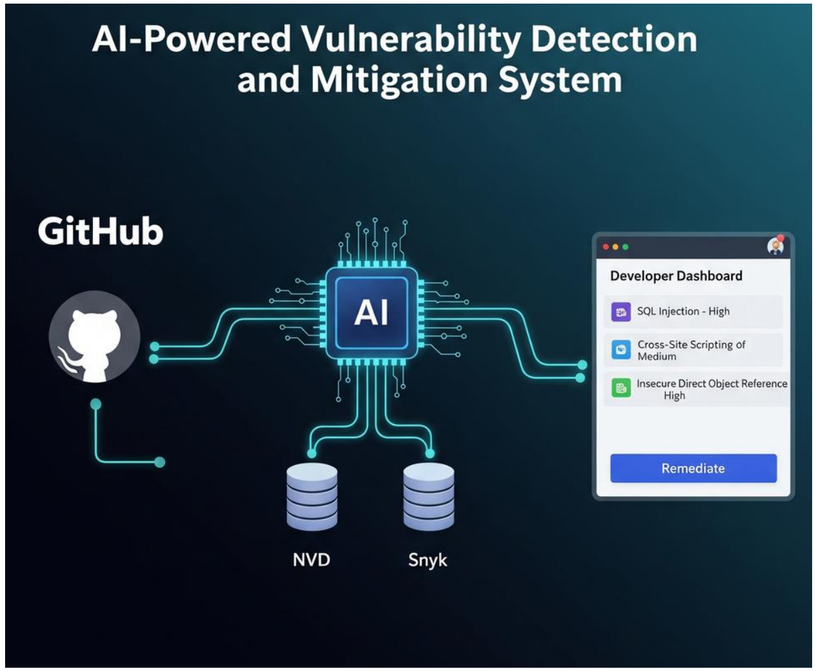
- Depend mainly on version-based vulnerability matching
- Large number of false positives
- No understanding of project context or exploitability
- Security teams face alert fatigue

Introduction

Proposed Solution
AI-Powered Vulnerability Detection and Mitigation System

Key Features:
- Automated dependency extraction from GitHub repositories
- Integration with NVD and Snyk vulnerability intelligence sources
- Vector embeddings and semantic similarity search
- LLM-based contextual vulnerability prioritization
- AI-generated remediation recommendations

Goal:
Improve vulnerability prioritization and assist developers in faster mitigation.



Problem Statement

Modern software applications depend heavily on third-party libraries and open-source packages. Traditional vulnerability scanning tools detect security flaws primarily through version-based CVE matching, which often generates a large number of alerts without considering the specific context of the software project.

As a result, developers and security teams face alert fatigue, inefficient vulnerability prioritization, and delayed remediation, increasing the risk of exploitation and security breaches.

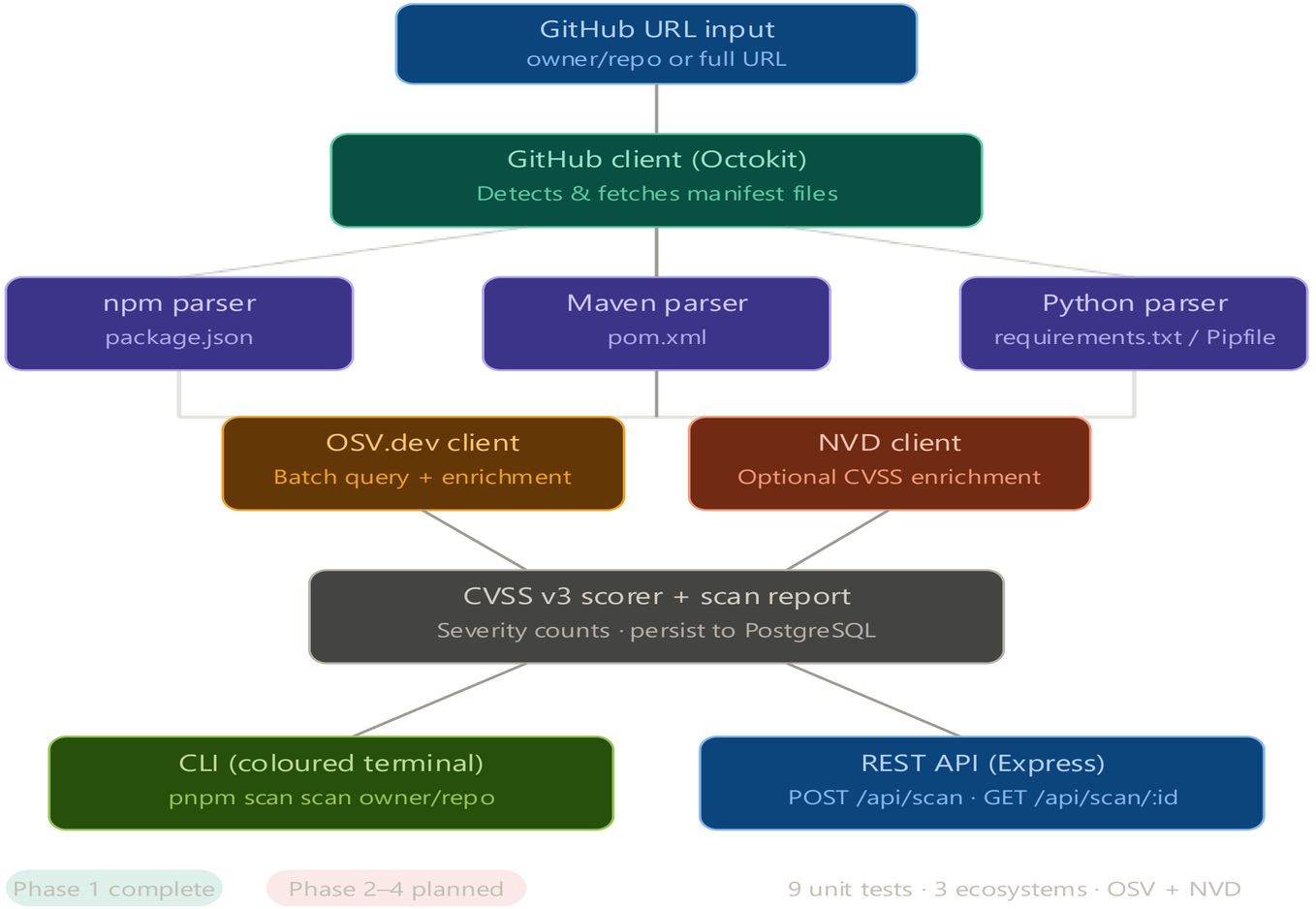
Objectives

- To design a system capable of extracting dependency information from repository manifest files such as package.json, pom.xml, and requirements.txt.
- To implement automated vulnerability detection by integrating trusted vulnerability intelligence sources such as NVD and Snyk.
- To enable context-aware vulnerability prioritization using vector embeddings and LLM-based reasoning rather than relying solely on CVSS severity scores.
- To generate AI-driven remediation strategies, including safe dependency upgrades and alternative package recommendations.
- To ensure developers receive structured vulnerability reports with prioritized risks and actionable mitigation guidance.

Literature Review

Author(s)	Year	Title	What they proposed	Method used	Key Result	Limitation	Research Gap Identified
Sabottke et al.	2015	Vulnerability Intelligence and Exploit Prediction	Studied relationship between vulnerability scores and real-world exploit likelihood	Machine learning analysis of vulnerability data	Found that CVSS scores alone are not reliable indicators of exploitation risk	Severity scores fail to capture contextual exploitability	Need intelligent systems that analyze vulnerability context beyond CVSS
Decan et al.	2018	On the Evolution of Technical Lag in Dependency Management	Studied how software projects depend on external libraries and analyzed dependency update delays	Large-scale empirical analysis of software repositories	Demonstrated that many projects use outdated dependencies, increasing security risk	Focuses on dependency updates but does not provide automated vulnerability mitigation	Need for automated systems to detect and prioritize vulnerable dependencies
Pashchenko et al.	2018	Vulnerability Scanners: A Study on False Positives	Evaluated effectiveness of vulnerability scanners like OWASP Dependency Check	Static vulnerability matching with CVE databases	Identified high number of vulnerability alerts in modern software projects	Many detected vulnerabilities are not exploitable in the specific project context	Need for context-aware vulnerability prioritization methods

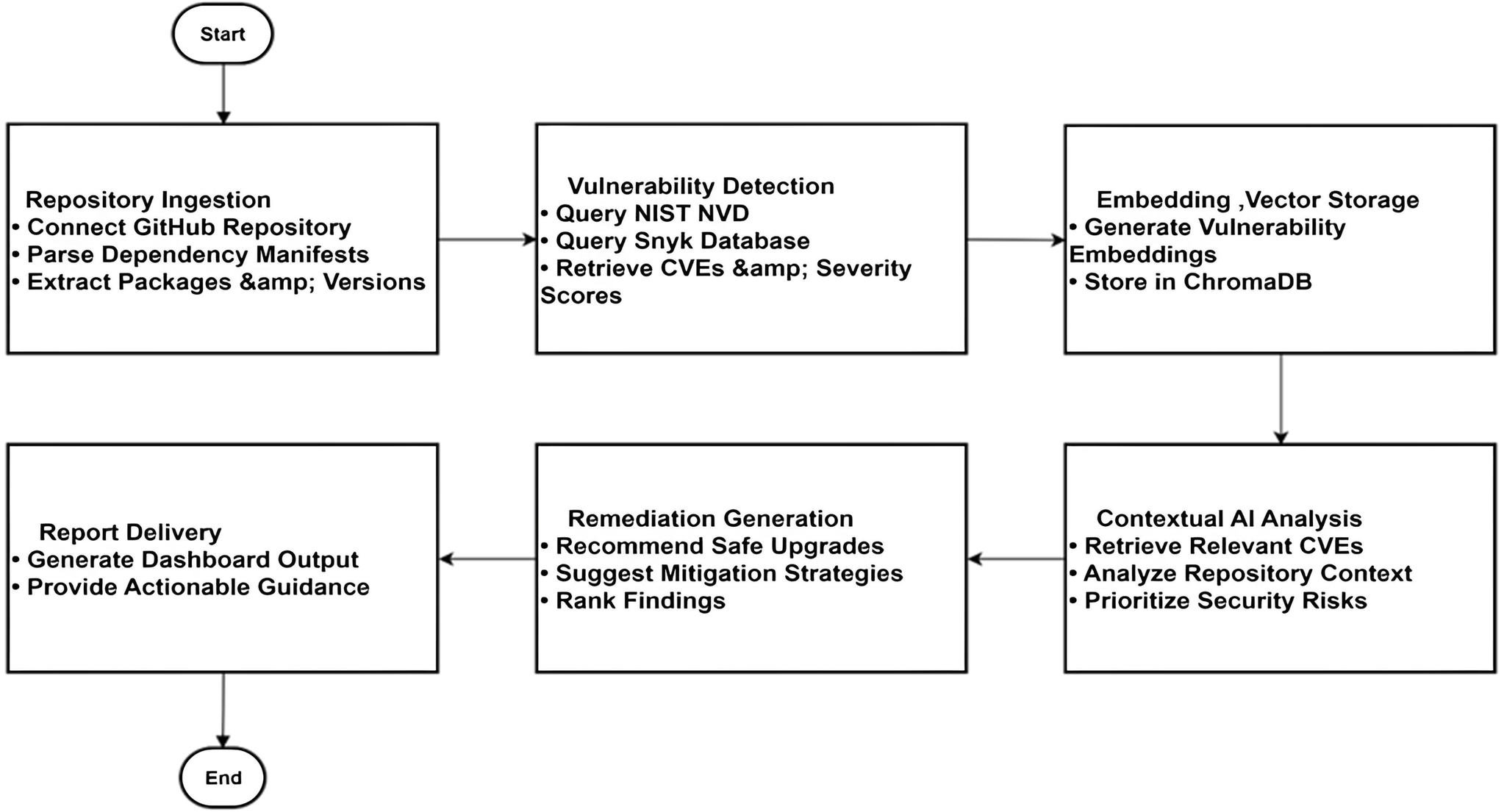
Methodology



Methodology

- **CLI Tool / REST Client** Provides two interfaces to trigger scans — a colour-coded terminal CLI for direct use and a REST API endpoint for programmatic access by external tools or future dashboards.
- **Express REST API** Acts as the core backend that receives scan requests, delegates to the scanning engine, and returns structured vulnerability reports as JSON. Built with Express 5 on Node.js.
- **GitHub Client (Octokit)** Connects to any public GitHub repository and automatically detects and downloads dependency manifest files from the root and common subdirectories.
- **Manifest Parsers** Extracts dependency names and versions from three ecosystems — package.json for npm, pom.xml for Maven, and requirements.txt / Pipfile for PyPI — using custom-built parsers with 9 unit tests.
- **OSV.dev API** Primary vulnerability source. Provides free, unauthenticated batch querying across 30+ advisory databases including GitHub Security Advisories and the Python advisory database. Up to 1,000 packages queried per request.
- **NIST NVD API** Secondary enrichment source. Provides official CVE records and richer CVSS v3 metadata for vulnerabilities where severity information was absent from OSV results. Rate-limited; API key recommended.
- **CVSS v3 Scorer** Computes base severity scores manually from CVSS vector strings using the official v3 formula. Classifies each vulnerability as Critical, High, Medium, Low, or Unknown and drives the colour-coded report output.

Methodology



Methodology

Repository Ingestion

- Connect to GitHub repository and extract dependency manifest files such as package.json, pom.xml, or requirements.txt.

Vulnerability Detection

- Match extracted dependencies with NVD and Snyk databases to identify associated CVEs and severity scores.

Embedding & Vector Storage

- Convert vulnerability descriptions into vector embeddings and store them in ChromaDB.

Contextual AI Analysis

- Use LLM-based reasoning to analyze repository context and prioritize security risks.

Remediation Generation

- Recommend safe dependency upgrades or alternative mitigation strategies.

Report Delivery

- Generate a dashboard report showing prioritized vulnerabilities and remediation guidance.

Expected Outcomes

Automated Vulnerability Detection

- The system identifies known vulnerabilities in third-party dependencies using CVE databases such as NVD and Snyk.

Context-Aware Risk Prioritization

- Vulnerabilities are ranked based on exploitability and repository context rather than relying only on CVSS scores.

AI-Based Remediation Suggestions

- Provides mitigation guidance such as secure dependency upgrades or alternative package recommendations.

Vulnerability Reporting Dashboard

- Displays structured reports with prioritized vulnerabilities and recommended mitigation actions.

Practical Demonstration

- System will demonstrate vulnerability detection and mitigation on test repositories containing known vulnerabilities.

Applications

Software Development Teams

- Helps developers identify vulnerable dependencies early and receive remediation suggestions during the development process.

DevSecOps Pipelines

- Can be integrated into CI/CD pipelines to perform automated vulnerability scanning and prioritization during software deployment.

Security Operations Centers (SOC)

- Assists security analysts in identifying high-risk vulnerabilities and focusing on the most critical security threats.

Software Supply Chain Security Research

- Can be used in academic and research environments to study dependency vulnerabilities and AI-based security analysis techniques.

Implementation – Phase 1

- **TypeScript Monorepo (pnpm workspaces)**
  - @vuln-shield/core — scanning engine
  - @vuln-shield/api — Express REST server
- **Scanning Engine**
  - Fetches manifests from GitHub via Octokit
  - Parses npm, Maven, PyPI dependencies
  - Queries OSV.dev batch API for CVE lookup
    - NIST NVD used for optional CVSS enrichment
  - CVSS v3 score computed manually from vector strings
  - Results stored in PostgreSQL via Prisma ORM
- **REST API**
  - POST /api/scan — triggers a scan
  - GET /api/scan/:id — fetch results
  - GET /api/scans — list scan history
- **Validated against** snyk-labs/nodejs-goof 35 deps · 58 vulnerabilities · 11 Critical

References

Research Papers / Journals

[1] Decan, A., Mens, T., & Constantinou, E., "On the Evolution of Technical Lag in Dependency Management," IEEE ICSME, 2018.

[2] Pashchenko, I., et al., "Vulnerability Scanners: A Study on False Positives," International Conference on Software Engineering (ICSE), 2018.

[3] Sabottke, C., Suciu, O., & Dumitraș, T.,"Vulnerability Intelligence and Exploit Prediction," USENIX Security Symposium, 2015.

[4] Zimmermann, T., et al., "A Large-Scale Study of Security Vulnerabilities in Open Source Software," IEEE Transactions on Software Engineering, 2019.